

MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)

Mini-Project Title: Mini-Project 2: Real-time Study Room Booking App (React Native & Expo)

Team / Student Name: Lê Cẩm

Submission Date: 05/10/2026

1. GENERAL INFORMATION & DELIVERABLE LINKS

- **Team Members:** Lê Cẩm — Student ID: 21IT001 — Role: Fullstack Mobile Architecture & Concurrency Engine — Contribution: 100%
-  **Live Demo URL:** <https://camle-vku-study-room.lecam.workers.dev>
-  **GitHub Repository:** <https://github.com/CAMLC25/camle-vku-study-room>
-  **Video Demo (Optional):** <https://camle-vku-study-room.lecam.workers.dev>

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	Responsive Viewport & Safe Area	 Complete	Custom hook useResponsiveLayout adapts across Mobile (1 col), Tablet (2 cols), Desktop (3 cols). Native safe area insets via react-native-safe-area-context.
2	Server State & Caching (TanStack Query)	 Complete	Integrated @tanstack/react-query with QueryClientProvider (staleTime: 5min, gcTime: 10min). Custom hook useRooms() with pull-to-refresh on FlatList.
3	Client State & Persistence (Zustand)	 Complete	Zustand store with persist middleware storing in @react-native-async-storage/async-storage under

key 'vku-booking-storage'.

4	Type-Safe Navigation (React Navigation 7)	 Complete	Nested Stack + Bottom Tabs architecture (RootNavigator + MainTabs) with strict TypeScript types (RootStackParamList, MainTabParamList).
5	Instant Search & Multi-Parameter Filter	 Complete	Real-time text search and filter chips by Building (A, B, C, V), Capacity (2–20), and Equipment (Projector, Whiteboard, High-spec PC, AC).
6	7-Day Rolling Horizon & Discrete Slots	 Complete	Interactive date selector for 7 days ahead; 4 discrete 2-hour daily sessions (07:30–09:30, 09:30–11:30, 13:00–15:00, 15:00–17:00).
7	90-Second Soft Hold & Realtime Sync	 Complete	Instant reservation lock with 90s countdown modal; broadcasts HELD_BY_OTHER amber state to other peers via Supabase Realtime channels.
8	Concurrency & Anti-Collision Engine	 Complete	Server-authoritative PostgreSQL advisory locking and partial unique index (idx_bookings_active_slot). Client-generated UUID tokens prevent replay duplicates.
9	Offline Outbox & Conflict Resolution	 Complete	Offline booking requests queued in an outbox (PENDING_SYNC) and flushed sequentially upon reconnection with deterministic conflict handling.

10 **QR Check-in Pass & Local Notifications**

✓
Complete

Interactive check-in pass powered by react-native-qrcode-svg; scheduled 15-minute countdown reminders via expo-notifications.

11 **Layout Animations (Reanimated 3/4)**

✓
Complete

Staggered entrance animations on room feed using FadeInDown.delay(index * 60).springify() and spring physics on interactions.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

3.1 Directory Organization

vku-study-room/

- | — App.tsx # Entrypoint with Safe Area, QueryProvider & RootNavigator
- | — app.json # Expo Managed configuration & asset definitions
- | — wrangler.jsonc # Cloudflare Workers Static Assets deployment config
- | — src/
 - | — components/ # Reusable UI components (RoomCard, SearchBar, FilterChips, Modal)
 - | — config/ # Environment variables and runtime mode detection
 - | — hooks/ # Custom hooks (useRooms, useResponsiveLayout, useRoomAvailability)
 - | — i18n/ # Multi-language dictionary (Vietnamese & English)
 - | — navigation/ # Type-safe navigators (RootNavigator, MainTabs, types.ts)
 - | — providers/ # TanStack QueryClientProvider configuration
 - | — screens/ # Core screens (RoomListScreen, RoomDetailScreen, MyBookingsScreen, ProfileScreen)
 - | — services/ # Abstraction layer (Supabase, Mock, Outbox Sync, Notifications)
 - | — store/ # Zustand stores (useBookingStore with persist, useAuthStore, useLanguageStore)
 - | — theme/ # Design tokens, typography, and VKU color palettes

```

|   └─ types/           # Strict TypeScript domain interfaces
|   └─ supabase/
|       └─ schema.sql    # PostgreSQL DDL, advisory lock RPC, and RLS policies
|       └─ seed.sql       # 20 pre-seeded study rooms across 4 VKU campus
buildings
└─ test/                # Node.js automated verification harness (Concurrency,
Outbox, Auth, QR)

```

3.2 State Management & Data Flow Architecture

- **Server State (TanStack Query):** Manages remote entity caches ('rooms') with a 5-minute staleTime, automated background synchronization, and zero-redundancy network refetching on pull-to-refresh.
- **Client State (Zustand):** Manages volatile and local domain state (active reservations, active 90s hold session, outbox queue, filter parameters, and user authentication). All critical mutations persist immediately to device storage via `@react-native-async-storage/async-storage` ('vku-booking-storage').

4. EMPIRICAL EVIDENCE & SCREENSHOTS

- **Room Discovery & Multi-Parameter Filter (RoomListScreen):** Displays 20 study rooms across Buildings A, B, C, and V in an adaptive 60 FPS grid. Features real-time filtering chips by building, seat capacity (2–20), and equipment tags with smooth staggered entry animations (`FadeInDown.springify()`).

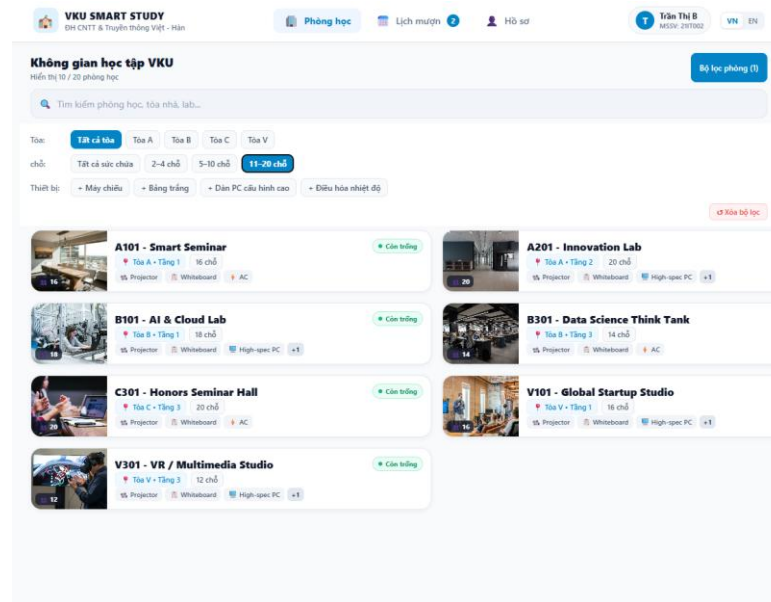


Figure 1. Room Discovery & Multi-Parameter Filter (RoomListScreen)

- Interactive Time-Slot Selector & 90s Countdown Modal (RoomDetailScreen):**
 Features a 7-day rolling calendar with 4 discrete two-hour slots per day. Selecting an available slot initiates a 90-second soft hold with a synchronized countdown timer, broadcasting HELD_BY_OTHER amber badges across all connected devices.

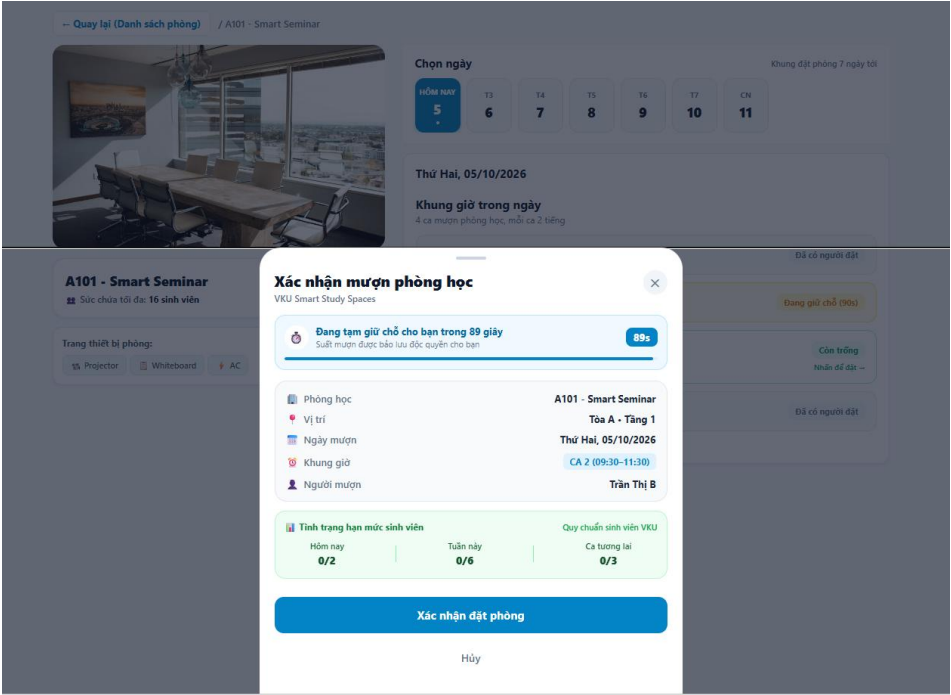


Figure 2. Interactive Time-Slot Selector & 90s Countdown Modal (RoomDetailScreen)

- Personal Reservations & Offline Synchronization (MyBookingsScreen):**
 Segmented tabs for 'All', 'Confirmed', 'Pending Sync', 'Conflicted', and 'Cancelled'. Displays local outbox queue status during offline mode and automatically resolves conflicts with optimistic feedback.

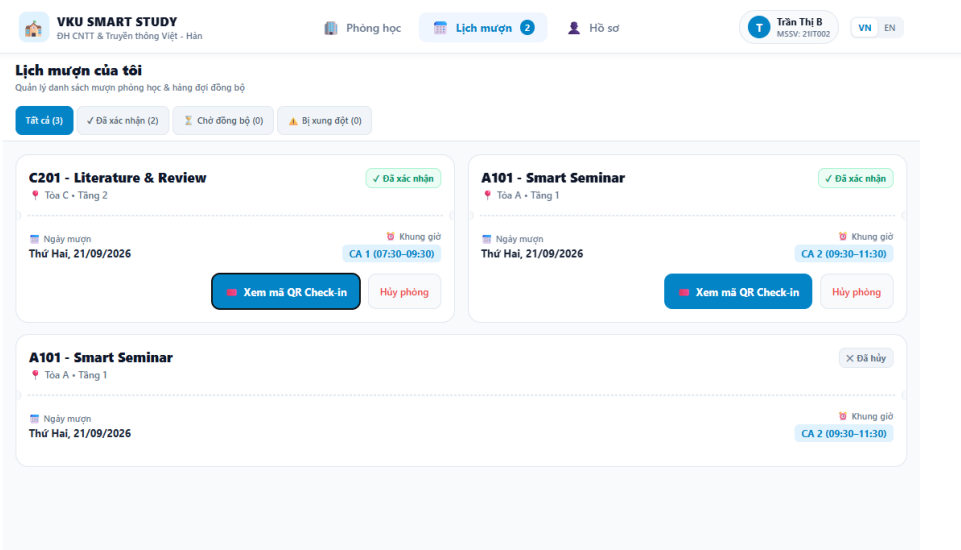


Figure 3. Personal Reservations & Offline Synchronization (MyBookingsScreen)

- **Digital QR Booking Pass (Check-in Modal):** Generates a high-contrast, cryptographically clean QR code encoding the unique booking token for campus security check-in.

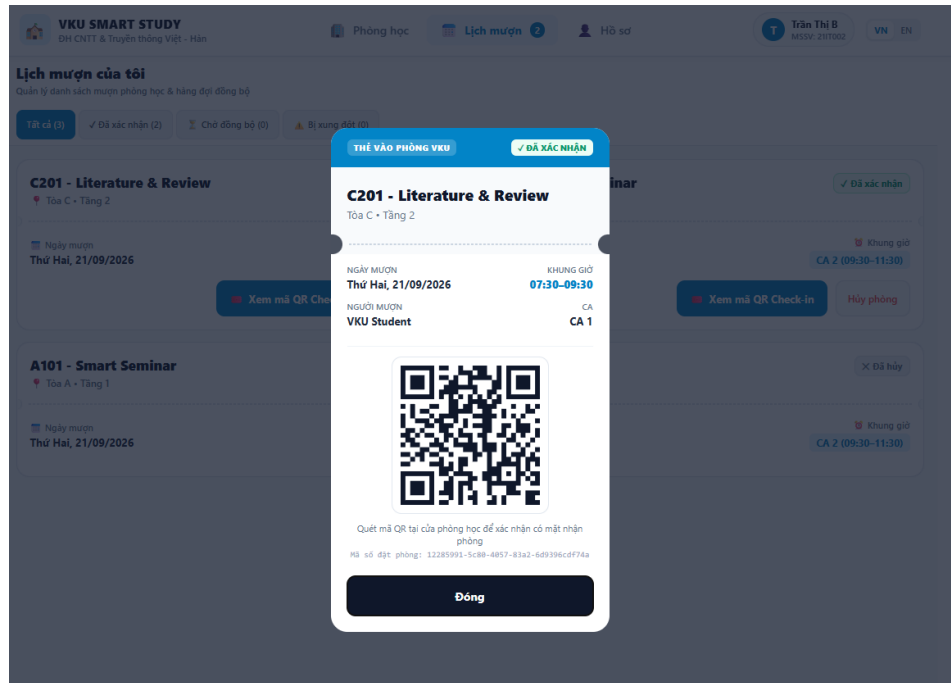


Figure 4. Digital QR Booking Pass (Check-in Modal)

5. TECHNICAL CHALLENGES & RESOLUTIONS

Challenge 1: Distributed Race Conditions & The 90-Second Soft Hold

Problem: In high-traffic scenarios (e.g., final exam week), multiple students may attempt to reserve the same room slot simultaneously. Naive client-side checks allow double-booking (Time-of-Check to Time-of-Use vulnerability). Furthermore, if a user dismissed the hold modal during network flight, unreleased holds caused phantom 90-second locks.

Resolution: (1) Implemented a dual-barrier locking mechanism: PostgreSQL transaction-level advisory locks (`pg_advisory_xact_lock`) combined with a partial unique constraint `idx_bookings_active_slot` (`room_id`, `booking_date`, `slot_index`) WHERE `status` = 'CONFIRMED'. (2) Implemented `isCancelledRef` inside `ConfirmBookingModal`: If the user cancels the modal before the server responds to `initiateHold`, the in-flight result is immediately rolled back via a background `releaseHold` call, ensuring slots return to AVAILABLE without delay.

Challenge 2: Offline Outbox Causality & Conflict Reconciliation

Problem: When a student books while walking through dead zones (offline), naive approaches either block the user or optimistically mark the booking as confirmed,

causing severe confusion if another student booked the slot online.

Resolution: Built an explicit Offline Outbox Engine: Offline bookings are created with state `PENDING_SYNC` and queued in FIFO order. When NetInfo detects reconnection, the engine sequentially flushes outbox items against the server. If the slot was taken while offline, the booking is transitioned deterministically to `CONFLICTED` with human-readable error diagnostics, avoiding silent data corruption.